

Texas College of Management & IT
Bachelor of Computer Science (BCS)

Assignment Title: Based on the Python Object-Oriented Programming (BCSNT 6063) syllabus, here is a comprehensive Mini Project guide. This guide includes 30 diverse themes, a structured marking scheme, and clear submission instructions. You can Pick any topic form provided theme for your project.

Submitted By:

Name: Prasanna Shrestha
LCID: LC00017004030

Year/Semester: BCS 1st Year / 2nd Semester

Submitted To:

Lecturer Name: Dr. Ashish Gautam
Course: Object Oriented Programming (BCSNT 6063)

Date of Submission:

7/26/2026

Texas College of Management & IT, Kathmandu

MINI PROJECT:

Supermarket billing system

[main.py](#):

:

```
main.py / ...  
from supermarket import Supermarket  
  
def menu():  
    market = Supermarket()  
  
    while True:  
        print("\n" + "=" * 40)  
        print("    SUPERMARKET BILLING SYSTEM")  
        print("=" * 40)  
        print("1. Add Product")  
        print("2. View Products")  
        print("3. Add To Cart")  
        print("4. Generate Bill")  
        print("5. Exit")  
  
        choice = input("Enter your choice: ")  
  
        try:  
            if choice == "1":  
                pid = input("Product ID: ")  
                name = input("Product Name: ")  
                price = float(input("Price: "))  
  
                market.add_product(pid, name, price)  
  
            elif choice == "2":  
                market.view_products()  
  
            elif choice == "3":  
                pid = input("Product ID: ")  
                qty = int(input("Quantity: "))  
  
                market.add_to_cart(pid, qty)  
  
            elif choice == "4":  
                market.generate_bill()
```

```

        elif choice == "5":
            print("Thank you!")
            break

        else:
            print("Invalid choice!")

    except ValueError:
        print("Invalid input! Please enter numeric values for price and quantity.")

if __name__ == "__main__":
    menu()

```

[product.py:](#)

```

from item import Item

class Product(Item):

    def __init__(self, product_id, name, price):
        self.__product_id = product_id
        self.__name = name
        self.__price = float(price)

    def get_id(self):
        return self.__product_id

    def get_name(self):
        return self.__name

    def get_price(self):
        return self.__price

    def display(self):
        print(f"{self.__product_id} | {self.__name} | Rs. {self.__price}")

```

[item.py:](#)

```

1  from abc import ABC, abstractmethod
2
3  class Item(ABC):
4
5      @abstractmethod
6      def display(self):
7          pass

```

supermarket.py:

```
import json
from product import Product

class Supermarket:
    def __init__(self):
        self.__products = []
        self.__cart = []
        self.load_products()

    def load_products(self):
        try:
            with open("products.json", "r") as file:
                data = json.load(file)

                for item in data:
                    self.__products.append(
                        Product(item["id"], item["name"], item["price"])
                    )

        except FileNotFoundError:
            pass

    def save_products(self):
        data = []

        for product in self.__products:
            data.append({
                "id": product.get_id(),
                "name": product.get_name(),
                "price": product.get_price()
            })

        with open("products.json", "w") as file:
            json.dump(data, file, indent=4)

    def add_product(self, pid, name, price):
        product = Product(pid, name, price)
        self.__products.append(product)
        self.save_products()
        print("Product added successfully!")
```

```
    def view_products(self):
        if not self.__products:
            print("No products available.")
            return

        print("\n===== PRODUCTS =====")

        for product in self.__products:
            product.display()

    def add_to_cart(self, pid, qty):
        for product in self.__products:
            if product.get_id() == pid:
                self.__cart.append((product, int(qty)))
                print("Added to cart!")
                return

        print("Product not found!")

    def generate_bill(self):
        if not self.__cart:
            print("Cart is empty.")
            return

        subtotal = 0

        print("\n===== BILL =====")

        for product, qty in self.__cart:
            total = product.get_price() * qty
            subtotal += total

            print(f"{product.get_name()} x{qty} = Rs. {total}")

        discount = subtotal * 0.10 if subtotal >= 5000 else 0
        tax = (subtotal - discount) * 0.13
        final = subtotal - discount + tax
```

```
    print("-----")
    print(f"Subtotal : Rs. {subtotal}")
    print(f"Discount : Rs. {discount}")
    print(f"VAT (13%): Rs. {tax}")
    print(f"Total : Rs. {final}")

    self.__cart.clear()
```

Product.json:

```
1  [
2    {
3      "id": "P101",
4      "name": "Rice",
5      "price": 1200.0
6    },
7    {
8      "id": "P102",
9      "name": "Oil",
10     "price": 3500.0
11   },
12   {
13     "id": "1291041",
14     "name": "Book",
15     "price": 5000.0
16   },
17   {
18     "id": "19280",
19     "name": "Book",
20     "price": 500.0
21   }
22 ]
```

Output:

```
=====
SUPERMARKET BILLING SYSTEM
=====
1. Add Product
2. View Products
3. Add To Cart
4. Generate Bill
5. Exit
Enter your choice: 1
Product ID: 19280
Product Name: Book
Price: 500
Product added successfully!
```

```
=====
SUPERMARKET BILLING SYSTEM
=====
1. Add Product
2. View Products
3. Add To Cart
4. Generate Bill
5. Exit
Enter your choice: 2

===== PRODUCTS =====
P101 | Rice | Rs. 1200.0
P102 | Oil | Rs. 3500.0
1291041 | Book | Rs. 5000.0
19280 | Book | Rs. 500.0
```

Explanation of main.py

The main.py file is the entry point of the Supermarket Billing System and controls the overall flow of the program. It imports the Supermarket class from supermarket.py, creates an object of the class, and displays a menu-driven interface using a while loop. The user can choose to add products, view products, add items to the shopping cart, generate a bill, or exit the program. Based on the user's choice, the corresponding method from the Supermarket class is called. If an invalid option is entered, the program displays an error message, making the application simple, interactive, and user-friendly.

Explanation of product.py

The product.py file contains the Product class, which represents an individual product in the supermarket. The class stores the product ID, name, and price as private attributes, demonstrating the concept of encapsulation in Object-Oriented Programming (OOP). It provides getter methods (get_id(), get_name(), and get_price()) to access these values safely and includes a display() method to print the product details in a formatted manner. This class acts as a blueprint for creating product objects that are used throughout the system.

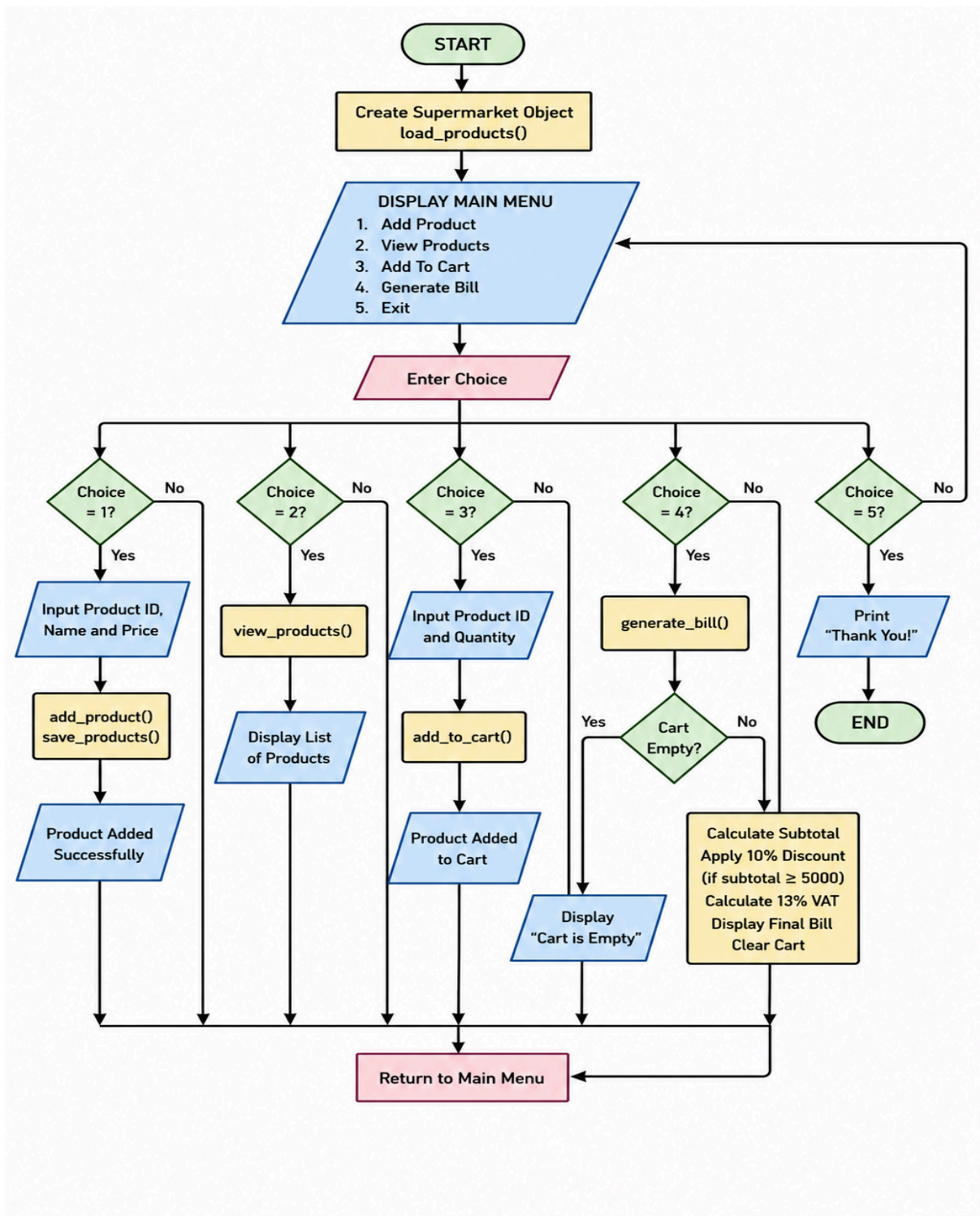
Explanation of supermarket.py

The supermarket.py file contains the Supermarket class, which manages all the core functionalities of the application. It loads products from the products.json file, allows users to add and view products, manages the shopping cart, and generates the final bill with discount and VAT calculations. The class also handles data persistence by saving product information back to the JSON file whenever a new product is added. Additionally, it uses exception handling with a try-except block to prevent the program from crashing if the product file does not exist, making the system more reliable.

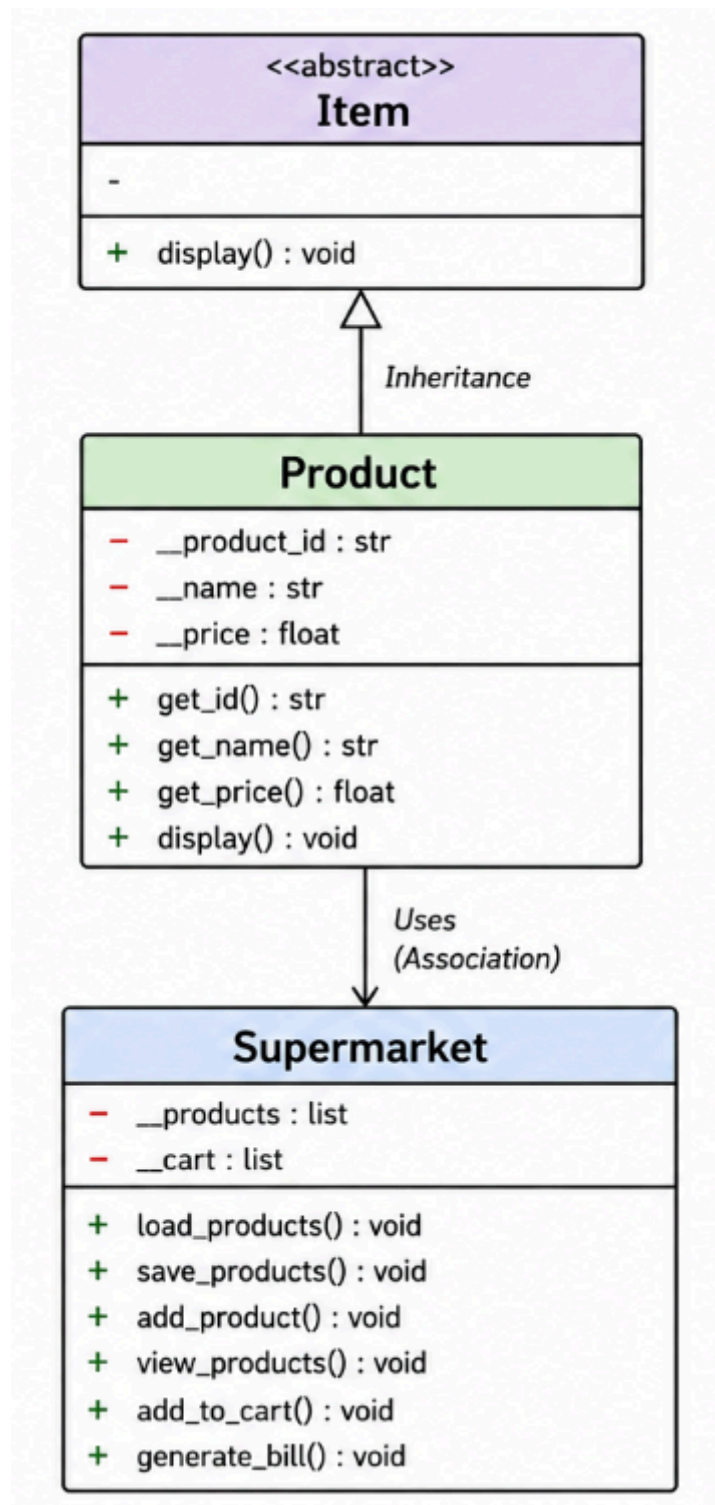
Explanation of products.json

The products.json file is used to store product information permanently in JSON format, ensuring that data is not lost when the application is closed. Each product is stored as a JSON object containing its ID, name, and price, making the file easy to read and update. When the program starts, the Supermarket class reads the file and loads all products into memory, and whenever a new product is added, the updated information is saved back to the file. This implementation satisfies the project's data persistence requirement and provides a simple method of storing product records.

FLOWCHART:



CLASS DIAGRAM:



FILE STRUCTURE:

```
|— main.py  
|— supermarket.py  
|— product.py  
|— item.py  
|— products.json  
|— README.md
```

CONCLUSION:

The Supermarket Billing System successfully demonstrates the application of Object-Oriented Programming (OOP) concepts to solve a real-world problem. The project provides a simple and user-friendly console interface that allows users to add products, view available items, manage a shopping cart, and generate bills with automatic discount and VAT calculations. It also stores product data in a JSON file, ensuring that information is retained even after the program is closed.

Furthermore, the project fulfills the required technical objectives by implementing Encapsulation through private attributes, Inheritance and an Abstract Base Class (ABC) for code reusability, data persistence using JSON, and exception handling to improve program reliability. Overall, this project enhanced the understanding of Python programming, file handling, and OOP principles while demonstrating how these concepts can be applied to develop a practical and maintainable software application.